

Distributed Mutual Exclusion Algorithms

Introduction

- Mutual exclusion: Concurrent access of processes to a shared resource or data is executed in mutually exclusive manner.
- Only one process is allowed to execute the critical section (CS) at any given time.
- Example – Directory management
- In a distributed system, shared variables (semaphores) cannot be used to implement mutual exclusion.
Message passing is the sole means for implementing distributed mutual exclusion.

Classification of Mutual Exclusion Algorithms

- Distributed mutual exclusion algorithms must deal with unpredictable message delays and incomplete knowledge of the system state.

Two basic approaches for distributed mutual exclusion:

Token based approach

Non-token based approach

- **Token-based approach:**
 - △ A unique token is shared among the sites.
 - △ A site is allowed to enter its CS if it possesses the token.
 - △ Mutual exclusion is ensured because the token is unique.

Non-token based approach:

- △ Two or more successive rounds of messages are exchanged among the sites to determine which site will enter the CS next.

Preliminaries

System Model

- The system consists of N sites, $S_1, S_2, \dots, S_N$.
- We assume that a single process is running on each site. The process at site S_i is denoted by p_i .

A site can be in one of the following three states: requesting the CS, executing the CS, or neither requesting nor executing the CS (i.e., idle).

- In the 'requesting the CS' state, the site is blocked and can not make further requests for the CS. In the 'idle' state, the site is executing outside the CS.

At any instant, a site may have several pending requests for CS. A site queues up these requests and serves them one at a time.

6.3.1 Requirements of Mutual Exclusion Algorithms

The primary objective of a mutual exclusion algorithm is to maintain mutual exclusion; that is, to guarantee that only one request accesses the CS at a time. In addition, the following characteristics are considered important in a mutual exclusion algorithm:

Freedom from Deadlocks. Two or more sites should not endlessly wait for messages that will never arrive.

Freedom from Starvation. A site should not be forced to wait indefinitely to execute CS while other sites are repeatedly executing CS. That is, every requesting site should get an opportunity to execute CS in a finite time.

Fairness. Fairness dictates that requests must be executed in the order they are made (or the order in which they arrive in the system). Since a physical global clock does not exist, time is determined by logical clocks. Note that fairness implies freedom from starvation, but not vice-versa.

Fault Tolerance. A mutual exclusion algorithm is fault-tolerant if in the wake of a failure, it can reorganize itself so that it continues to function without any (prolonged) disruptions.

Performance

Metrics

The performance is generally measured by the following four metrics:

- Message complexity: The number of messages required per CS execution by a site.
- Synchronization delay: After a site leaves the CS, it is the time required and before the next site enters the CS (see Figure 1).

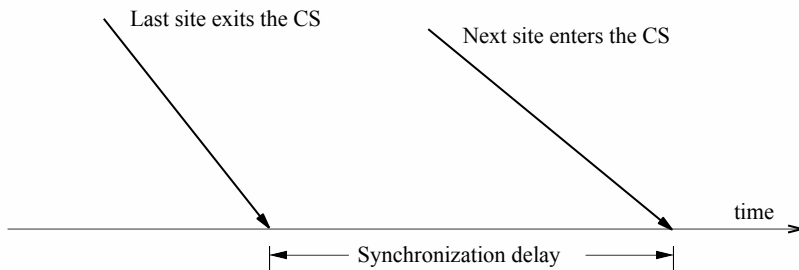


Figure 1: Synchronization Delay.

Performance

Metrics

- Response time: The time interval a request waits for its CS execution to be over after its request messages have been sent out (see Figure 2).

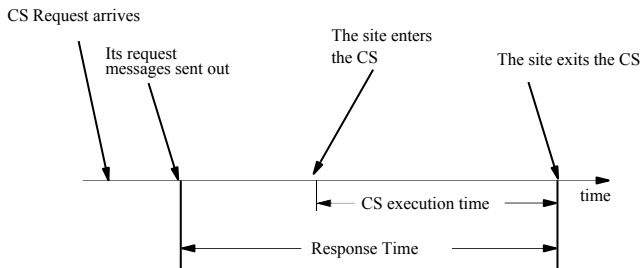


Figure 2: Response Time.

- System throughput: The rate at which the system executes requests for the CS.

$$\text{system throughput} = 1 / (SD + E)$$

where SD is the synchronization delay and E is the average critical section execution time.

Performance Metrics

Low and High Load Performance:

- We often study the performance of mutual exclusion algorithms under two special loading conditions, viz., “low load” and “high load”.
- The load is determined by the arrival rate of CS execution requests.
- Under *low load* conditions, there is seldom more than one request for the critical section present in the system simultaneously.
- Under *heavy load* conditions, there is always a pending request for critical section at a site.

Non-Token- Based Algorithm

Lamport's Algorithm

- Requests for CS are executed in the increasing order of timestamps and time is determined by logical clocks.
- For a site S_i , *request set* R_i contain ids of all those sites from which S_i must acquire permission before entering the CS.
- Every site S_i keeps a queue, *request queue*, which contains mutual exclusion requests ordered by their timestamps.

This algorithm requires communication channels to deliver messages the FIFO order.

The Algorithm

Requesting the critical section:

- When a site S_i wants to enter the CS, it broadcasts a $\text{REQUEST}(ts_i, i)$ message to all other sites and places the request on *request queue_i*. ((ts_i, i) denotes the timestamp of the request.)
- When a site S_j receives the $\text{REQUEST}(ts_i, i)$ message from site S_i , places site S_i 's request on *request queue_j* and it returns a timestamped REPLY message to S_i .

Executing the critical section: Site S_i enters the CS when the following two conditions hold:

- L1:** S_i has received a message with timestamp larger than (ts_i, i) from all other sites.
- L2:** S_i 's request is at the top of *request queue_i*.

The Algorithm

Releasing the critical section:

- Site S_i , upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.

When a site S_j receives a RELEASE message from site S_i , it removes S_i 's request from its request queue.

When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

Example

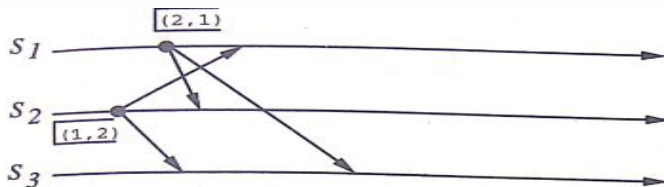
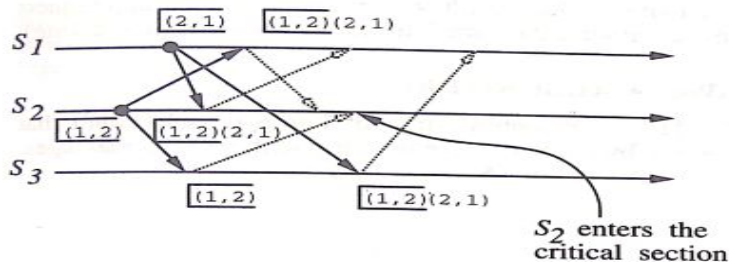


FIGURE 6.3

Sites S_1 and S_2 are making requests for the CS.



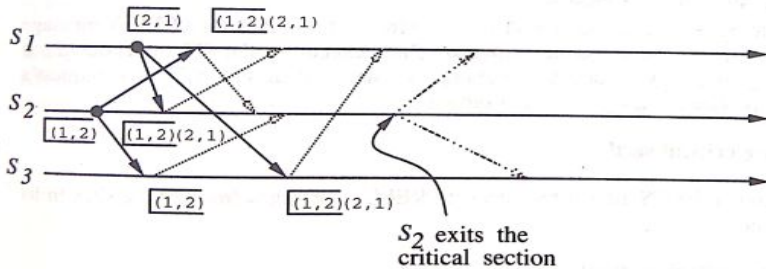
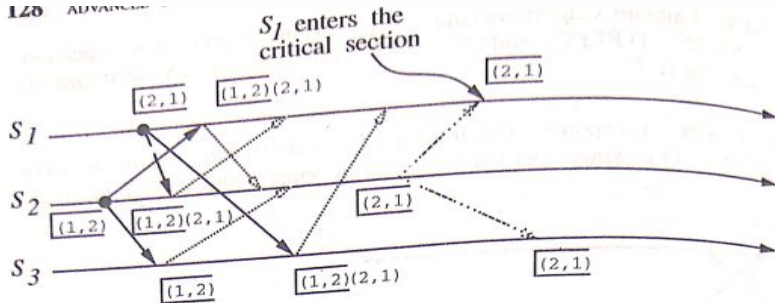


FIGURE 6.5
Site S_2 exits the CS and sends RELEASE messages.

120 ADVANCED



Performance

- For each CS execution, Lamport's algorithm requires $(N - 1)$ REQUEST messages, $(N - 1)$ REPLY messages, and $(N - 1)$ RELEASE messages.
- Thus, Lamport's algorithm requires $3(N - 1)$ messages per CS invocation.
- Synchronization delay in the algorithm is T .

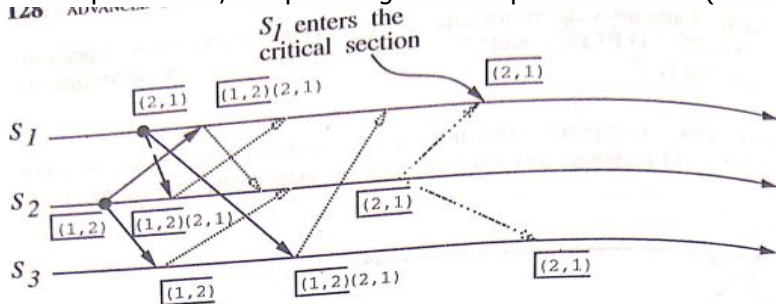
An

optimization

In Lamport's algorithm, REPLY messages can be omitted in certain situations. For example, if site S_j receives a REQUEST message from site S_i after it has sent its own REQUEST message with timestamp higher than the timestamp of site S_i 's request, then site S_j need not send a REPLY message to site S_i .

This is because when site S_j receives site S_i 's request with timestamp higher than its own, it can conclude that site S_i does not have any smaller timestamp request which is still pending.

With this optimization, Lamport's algorithm requires between $3(N - 1)$ at



Ricart-Agrawala Algorithm

- The Ricart-Agrawala algorithm assumes the communication channels are FIFO. The algorithm uses two types of messages: REQUEST and
- REPLY.

A process sends a REQUEST message to all other processes to request their permission to enter the critical section. A process sends a REPLY

- message to a process to give its permission to that process.

Processes use Lamport-style logical clocks to assign a timestamp to critical section requests and timestamps are used to decide the priority of requests.

Description of the Algorithm

Requesting the critical section:

- (a) When a site S_i wants to enter the CS, it broadcasts a timestamped REQUEST message to all other sites.
- (b) When site S_j receives a REQUEST message from site S_i , it sends a REPLY message to site S_i if site S_j is neither requesting nor executing the CS, or if the site S_j is requesting and S_i 's request's timestamp is smaller than site S_j 's own request's timestamp. Otherwise, the reply is deferred.

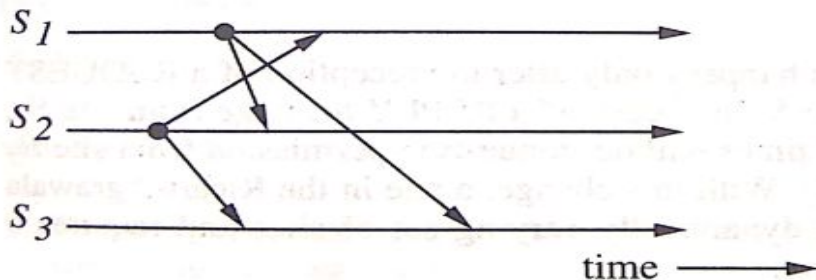
Executing the critical section:

- (c) Site S_i enters the CS after it has received a REPLY message from every site it sent a REQUEST message to.

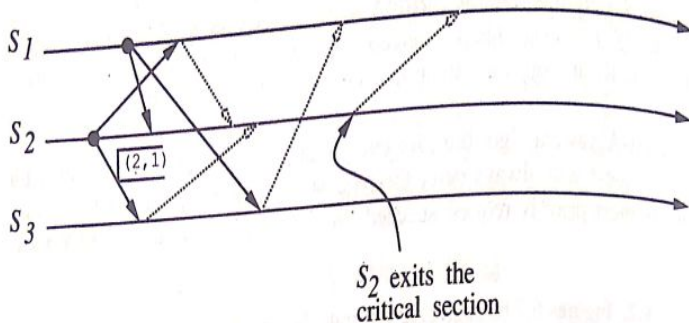
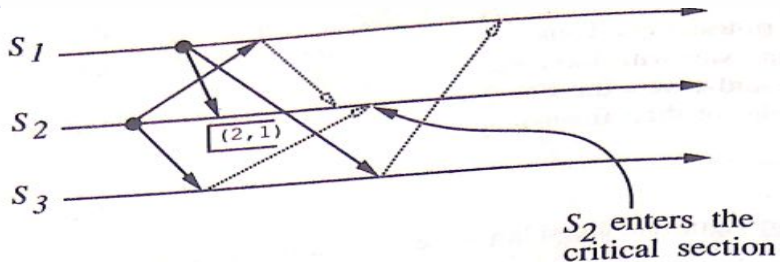
Algorithm

Releasing the critical section:

(d) When site S_i exits the CS, it sends all the deferred REPLY messages:

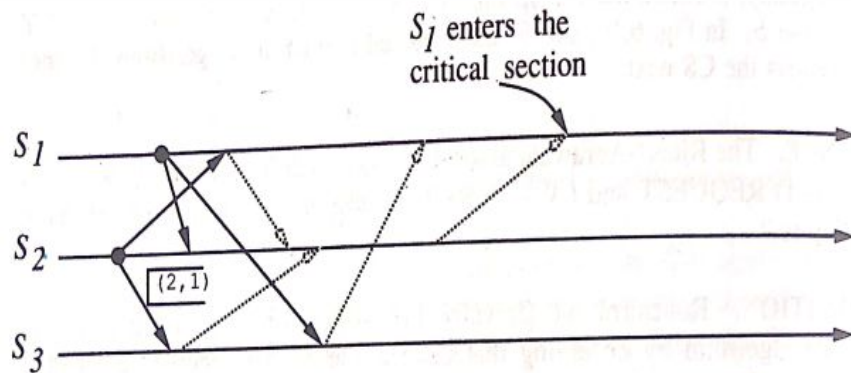


Algorithm



Algorithm

m



Performance

- For each CS execution, Ricart-Agrawala algorithm requires $(N - 1)$ REQUEST messages and $(N - 1)$ REPLY messages.
- Thus, it requires $2(N - 1)$ messages per CS
- execution. Synchronization delay in the algorithm is T .

Quorum-Based Mutual Exclusion Algorithms

Quorum-based mutual exclusion algorithms are different in the following two ways:

- 1 A site does not request permission from all other sites, but only ~~subset~~ ^{subset} of the sites. The request set of sites are chosen such that $\forall i \nexists j : 1 \leq i, j \leq N :: R_i \cap R_j \neq \Phi$. Consequently, every pair of sites which mediates conflicts between that pair has a site
- 2 A site can send out only one REPLY message at any time. A site can send a REPLY message only after it has received a RELEASE message for the previous REPLY message.

continuation.

Since these algorithms are based on the notion of 'Coterie' and 'Quorums', we next describe the idea of coterie and quorums. A coterie C is defined as a set of sets, where each set $g \in C$ is called a quorum. The following properties hold for quorums in a coterie:

- Intersection property: For every quorum $g, h \in C$, $g \cap h \neq \emptyset$.
For example, sets $\{1,2,3\}$, $\{2,5,7\}$ and $\{5,7,9\}$ cannot be quorums in a coterie because the first and third sets do not have a common element.

Minimality property: There should be no quorums g, h in coterie C such that $g \supseteq h$. For example, sets $\{1,2,3\}$ and $\{1,3\}$ cannot be quorums in a coterie because the first set is a superset of the second.

continuation.

Coterie and quorums can be used to develop algorithms to ensure mutual exclusion in a distributed environment. A simple protocol works as follows:

- Let 'a' is a site in quorum 'A'. If 'a' wants to invoke mutual exclusion, it requests permission from all sites in its quorum 'A'.

Every site does the same to invoke mutual exclusion. Due to the Intersection Property, quorum 'A' contains at least one site that is

- common to the quorum of every other site.

These common sites send permission to only one site at any time. Thus, mutual exclusion is guaranteed.

Maekawa's Algorithm

Maekawa's algorithm was the first quorum-based mutual exclusion algorithm. The request sets for sites (i.e., quorums) in Maekawa's algorithm are constructed to satisfy the following conditions:

M1: $(\forall i \ \forall j : i \neq j, 1 \leq i, j \leq N :: R_i \cap R_j \neq$

$\emptyset)$ **M2:** $(\forall i : 1 \leq i \leq N :: S_i \in R_i)$

M3: $(\forall i : 1 \leq i \leq N :: |R_i| = K)$

M4: Any site S_i is contained in K number of R_j s, $1 \leq i, j \leq N$.

Maekawa used the theory of projective planes and showed

that $N = K(K - 1) + 1$. This relation gives $|R_i| = \sqrt{N}$.

continuation.

-
- Conditions M1 and M2 are necessary for correctness; whereas conditions M3 and M4 provide other desirable features to the algorithm.
- Condition M3 states that the size of the requests sets of all sites must be equal implying that all sites should have to do equal amount of work to invoke mutual exclusion.
- Condition M4 enforces that exactly the same number of sites should request permission from any site implying that all sites have “equal responsibility” in granting permission to other sites.

The Algorithm

A site S_i executes the following steps to execute the CS.

Requesting the critical section

- (a) A site S_i requests access to the CS by sending $\text{REQUEST}(i)$ messages to all sites in its request set R_i .
- (b) When a site S_j receives the $\text{REQUEST}(i)$ message, it sends a $\text{REPLY}(j)$ message to S_i provided it hasn't sent a REPLY message to a site since its receipt of the last RELEASE message. Otherwise, it queues up the $\text{REQUEST}(i)$ for later consideration.

Executing the critical section

- (c) Site S_i executes the CS only after it has received a REPLY message from every site in R_i .

The Algorithm

Releasing the critical section

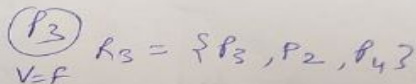
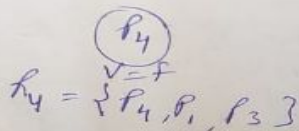
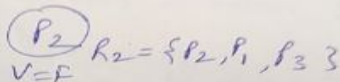
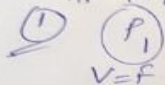
- (d) After the execution of the CS is over, site S_i sends a $\text{RELEASE}(i)$ message to every site in R_i .
- (e) When a site S_j receives a $\text{RELEASE}(i)$ message from site S_i , it sends a REPLY message to the next site waiting in the queue and deletes that entry from the queue. If the queue is empty, then the site updates its state to reflect that it has not sent out any REPLY message since the receipt of the last RELEASE message.

Performance

- Since the size of a request set is $\sqrt{N}$, an execution of the CS requires $\sqrt{N}$ REQUEST, $\sqrt{N}$ REPLY, and $\sqrt{N}$ RELEASE messages, resulting in $3\sqrt{N}$ messages per CS execution.

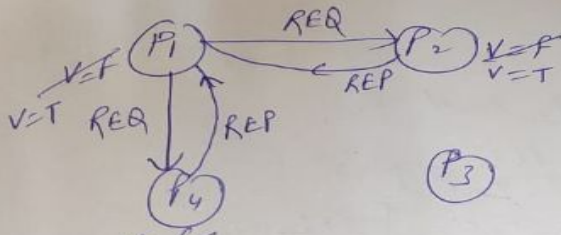
Example of Maekawa's :-

$$h_1 = \{P_1, P_2, P_4\}$$



②

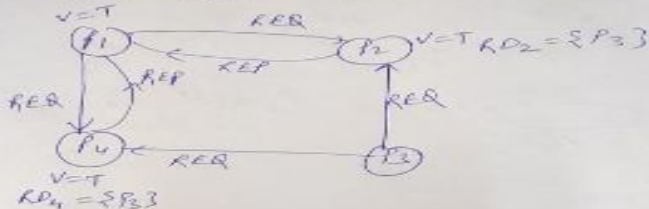
Now P_1 wants to enter in CS, so it will send request to P_2 and P_4 .



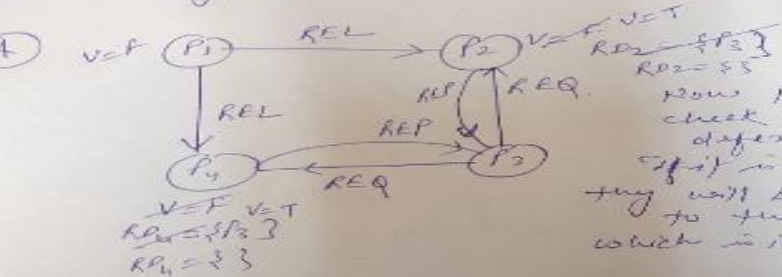
both P_2 and P_4 have $V=F$, so they will send reply to P_1 (if any process has $V=T$, it will not send reply msg to P_1).

(5)

Now P_3 want to execute CS, so it send req. to its member.



$\rightarrow P_1$ wants to leave CS, so it send Release msg to its member. Also it will reset V .



Now P_2 and P_4 check their deferred req. If it is not empty, they will send REP to the process which is in deferred.

Now P_3 will execute CS, as it has received lock from both P_2 and P_4 .

Problem of Deadlocks

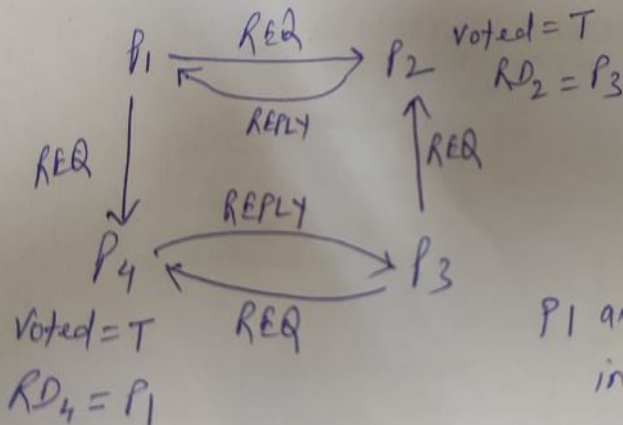
- Maekawa's algorithm can deadlock because a site is exclusively locked by other sites and requests are not prioritized by their timestamps.
- Assume three sites S_i , S_j , and S_k simultaneously invoke mutual exclusion. Suppose $R_i \cap R_j = \{S_{ij}\}$, $R_j \cap R_k = \{S_{jk}\}$, and $R_k \cap R_i = \{S_{ki}\}$.

Consider the following scenario:

- S_{ij} has been locked by S_i (forcing S_j to wait at S_{ij}).
- S_{jk} has been locked by S_j (forcing S_k to wait at S_{jk}).
- S_{ki} has been locked by S_k (forcing S_i to wait at S_{ki}).

This state represents a deadlock involving sites S_i , S_j , and S_k .

Problem of Deadlocks



P_1 and P_3 both are in deadlock.